OList E-Commerce ETL Pipeline Project

Introduction

Executive Summary

Olist Store, the largest department store in Brazil’s online marketplaces, serves as a key intermediary connecting thousands of small and medium-sized businesses to customers nationwide. This project analyzes a comprehensive dataset of approximately 100,000 orders from 2016 to 2018.

The primary objective is to derive **actionable insights into Olist's logistics performance**—focusing on delivery efficiency, on-time rates, and customer satisfaction. By integrating multiple datasets within a **Medallion Lakehouse architecture on Microsoft Fabric**, the analysis tracks the entire customer journey from purchase to delivery and review.

Through data transformation, KPI modeling, and dashboard visualization, the project identifies **key bottlenecks and patterns** such as **regional delivery delays**, **seller-specific performance gaps**, and **correlations between delivery time and customer ratings**. The insights generated will inform **strategic recommendations** to enhance operational efficiency, and ultimately elevate the customer experience.

Project Ideation

To ensure focus and analytical depth, we established clear criteria for choosing the problem:

- The challenge must require **problem-solving**, not just visualization.
- It should involve **ETL complexity** (joins, transformations, derived metrics).
- It must be **quantifiable and measurable** with KPIs.
- It should form **one cohesive narrative** that supports multi-member collaboration.

Starting Point: A Customer-First Perspective

Customer reviews are the most visible reflection of satisfaction in e-commerce.

A single rating encapsulates the entire shopping journey — from browsing to delivery — into one score that directly shapes both **brand reputation and revenue**.

Yet behind every low rating lies a chain of problems waiting to be uncovered.

With a **customer-first mindset**, we began with a broad problem statement:

> **"How can we increase customer ratings?"**

To explore this, we visualized the experience through the lens of a **typical Olist customer**.

Customer Persona: Meet Ana

Name: Ana Souza
Age: 32
Occupation: Working professional and mother of two
Location: São Paulo, Brazil
Shopping Habits: Frequently shops online for convenience, values reliability, clear

communication, and punctual delivery.

Expectations:

- Seamless shopping experience from order to delivery
- Accurate product information and tracking updates
- Receiving items on time

Frustrations:

- Delayed deliveries, especially for gifts or essentials
- Lack of delivery status visibility
- Wrong items, incomplete delivery.
- Product does not match description on web page
- Damaged product
- Unhelpful or delayed customer support when things go wrong

Ana's voice: "I don't mind waiting a few days, but at least tell me where my parcel is. When it arrives late without notice, I lose trust in the seller."

This persona guided our design of the **customer journey map**, ensuring we viewed every touchpoint through Ana's experience — not just as data points.

Mapping the Customer Journey

We analyzed the customer journey across eight key stages, aligning **expectations**, **pain points**, and **data visibility**.

Stage	Customer Expectations	Common Pain Points
Awareness & Browsing	Easy discovery, trusted info, user-friendly interface	Poor search, misleading info, confusing layout
Consideration	Clear product details, fair pricing	Hidden costs, inconsistent quality
Purchase & Payment	Smooth checkout, multiple payment modes	Payment failures, long checkout

Fulfillment & Dispatch	Prompt processing, stock accuracy	Seller delays, stock-outs
Logistics & Delivery	Fast shipping, accurate tracking, on-time arrival	Delays, missed dates, poor tracking
Product Receipt	Correct item, good packaging	Wrong or damaged items, poor packaging
After-Sales	Easy returns, quick refunds	Complicated returns, refund delays
Loyalty	Rewards, recognition	No loyalty program, ignored complaints

At each stage, we asked:

1. What problems could occur here?
2. How visible is this problem to customers like Ana?
3. Do we have enough data to measure and analyze it?

This process surfaced several potential focus areas

- Product quality (damaged or incorrect items)
- Packaging and presentation (poorly packed or misleading visuals)
- Customer support (slow or unresolved complaints)
- **Delivery performance (delays, long lead times, missed promised dates)**

Among these, **logistics and delivery performance** stood out as the **most measurable, high-impact, and customer-relevant** area.

Why We Chose the Logistics Angle

Our decision to focus on logistics was guided by **four key factors**:

1. Customer-Centric Perspective

For customers like Ana, delivery is the **moment of truth** in the e-commerce experience.

While product quality and packaging are important, **delivery reliability also influences satisfaction**.

Late or missed deliveries erode trust, whereas on-time arrivals create confidence and repeat purchases.

Thus, improving delivery performance aligns directly with enhancing **customer satisfaction and loyalty**.

2. Data Availability and Measurability

The Olist dataset provides **rich timestamp data** for each logistics milestone:

- `order_purchase_timestamp`
- `order_approved_at`
- `order_delivered_carrier_date`
- `order_delivered_customer_date`
- `order_estimated_delivery_date`

These allow us to compute precise metrics such as:

- **On-Time Delivery (OTD%)**
- **Delivery Promise Accuracy (DPA%)**
- **Lead Time and Cycle Time**
- **Order Processing and Transit Duration**

Other journey stages (e.g., product quality or customer support) lacked structured data for quantitative analysis. Logistics, by contrast, offered the **strongest data foundation** for meaningful insights.

3. Business Relevance

Efficient logistics is a **strategic differentiator** for e-commerce success. Delivery reliability affects not only customer satisfaction but also **seller reputation and operational cost**.

By analyzing logistics performance, our project provides **actionable guidance** to:

- Identify bottlenecks across approval, dispatch, and delivery
- Reduce delays and improve service-level adherence

- Suggest operational strategies such as adding **delivery hubs in underperforming regions**

4. Industry and Career Relevance

Supply chain and logistics analytics are among the **fastest-growing domains** in the data profession, making this project valuable for portfolio relevance.

By tackling this topic, our project bridges **data engineering, business intelligence, and domain understanding**, demonstrating our ability to translate data into real-world operational improvements.

Customer reviews are the most visible reflection of satisfaction in e-commerce. A single rating aggregates the entire shopping journey—from browsing to delivery—into a score that influences both reputation and revenue. Yet behind every low rating lies a set of problems waiting to be uncovered.

With a customer-first mindset, we started with a broad customer-first problem statement: **“How can we increase customer ratings?”**.

The next step was to map the customer journey and identify pain points.

Stage	Expectations	Pain Points
Awareness & Browsing	Easy discovery, trusted info, easy-to-use interface	Poor search, misleading info, confusing interface
Consideration	Clear and reasonable/ competitive pricing, product details	Hidden costs, inconsistent quality
Purchase & Payment	Smooth checkout, multiple payment options	Payment failures, long checkout
Fulfillment & Dispatch	Prompt processing, stock accuracy	Seller delays, stock-outs

Logistics & Delivery	Fast shipping, accurate tracking, on-time	Delays, missed dates, poor tracking, missing parcel
Product Receipt	Correct item, good packaging	Wrong/damaged items, bad packaging
After-Sales	Easy returns, quick refunds	Complicated returns, refund delays
Loyalty	Rewards, recognition	No loyalty program, ignored complaints

At each stage, we asked:

- What problems could occur here?
- How visible is this problem to customers?
- Do we have enough data to measure and analyze it?

This process surfaced multiple potential focus areas:

- Product quality (damaged or incorrect items)
- Packaging & presentation (poorly packed items, misleading product images)
- Customer support (slow responses, unresolved complaints)
- Delivery performance (late deliveries, long lead times, missed promised dates)

From the broader map, we choose to focus on **logistics and delivery performance.** This decision was guided by three main factors:

1. Customer-Centric Perspective

Delivery speed and reliability are among the most tangible aspects of customer experience. Unlike product quality, logistics performance is felt by nearly every customer. Delays or missed promised dates directly impact satisfaction and are often the top reason behind poor reviews.

2. **Data Availability and Measurability**

The Olist dataset provides rich timestamp fields for each logistics milestone: purchase date, approval date, shipping date, delivery date, and estimated delivery date. This allowed us to

calculate common metrics in logistics llike: On-time delivery, lead cycle time, order processing time, etc.

In contrast, areas like product quality or packaging lacked sufficient structured data for deep analysis.

3. **Business and Industry Relevance**

Efficient logistics is a core competitive differentiator in e-commerce. By focusing on this angle, our project connects operational efficiency directly to customer satisfaction.

Additionally, logistics and supply chain analytics are fast-growing fields in the data industry, making this project valuable for both portfolio and career relevance.

Project Objective

By focusing on logistics, our project aims to:

- Diagnose inefficiencies across approval, dispatch, and delivery stages
- Quantify delays and on-time performance
- Generate actionable recommendations for improving logistics efficiency
- Strengthen the link between operational excellence and customer satisfaction

Improvements in this domain translate into greater customer trust, reduced churn, and stronger marketplace competitiveness.

ETL/ELT Pipeline Development:
Create a robust data ingestion pipeline to load the dataset into a lakehouse with an appropriate schema design.

Data Exploration & Quality Assessment:
Perform extensive exploratory data analysis (EDA) to evaluate data quality and apply necessary transformations and preparations.

BI Dashboard Implementation:
Design and develop an interactive business intelligence dashboard using Power BI to visualize key metrics.

Functional Testing & Validation:
Conduct functional testing by comparing database query outputs with the dashboard reports to ensure consistency and accuracy.

Strategic Recommendations:

Generate strategic business recommendations to enhance Olist's logistics performance.

Domain Understanding

1. Logistics in the E-Commerce Context

In e-commerce, logistics represents the **movement of goods from sellers to customers**, encompassing multiple stakeholders **(seller → Olist → carrier → customer)** and data events. It is the bridge between **customer expectations** and **operational performance**. Any breakdown in this chain—delays, inaccurate tracking, lost parcels—directly affects customer satisfaction and reviews.

2. Logistics Flow and Data Events

Olist's marketplace logistics flow typically follows these stages:

1. **Order Placement** → customer confirms purchase.
2. **Payment Approval** → Olist or payment gateway authorizes payment.
3. **Seller Fulfillment** → seller prepares and packages order.
4. **Carrier Pickup** → courier collects parcel from seller/warehouse.
5. **In-Transit** → parcel travels between hubs.
6. **Delivery to Customer** → parcel arrives at customer location.

These events form the **core temporal backbone** of the dataset and enable the derivation of key metrics such as cycle times, delays, and SLA adherence.

3. Stakeholders and Their Goals

Stakeholder	Role	Logistics Goal / Concern
Customer	Expects reliable, fast delivery	Wants real-time tracking, accurate estimated delivery, minimal delays

Seller	Handles packaging and dispatch	Needs efficient pickup and clear SLA expectations
Carrier (3PL)	Transports parcels	Aims to optimize routing, meet OTD targets, avoid bottlenecks
Olist Platform	Oversees ecosystem	Must balance customer satisfaction with operational cost and SLA compliance
Data Team / Logistics Manager	Monitors KPIs	Uses analytics to detect delays, forecast capacity, improve SLA performance

Understanding these actors helps design a data model that reflects **real-world accountability (e.g., who caused a delay — seller or carrier?).**

4. Core Metrics Used In Logistics Industry

1. On-Time Delivery (OTD %)

```
OTD % = ( Number of delivered orders where delivered_date ≤ estimated_date ÷ Total delivered orders with both dates ) × 100
```

Explanation (Logistics View):
This is the **SLA compliance metric**. It tells us what percentage of shipments were delivered on or before the promised date.

- High OTD (≥95%) → operations are reliable, customers trust us.
- Low OTD → broken promises → customer complaints, refunds, churn.
- *Use case:* Daily/weekly monitoring against service-level targets.

2. Delivery Promise Accuracy (DPA %)

```
DPA % = ( Number of delivered orders where |delivered_date - estimated_date| ≤ tolerance_days ÷ Total delivered orders with both dates ) × 100
```

Explanation:
DPA checks if our **promises are realistic**. Even if we miss the exact day, are we close?

- Example: If we promised 5 days but delivered in 6 → still accurate (within ±1 day tolerance).
- High DPA = good forecasting → customers can plan better
- Low DPA = our promise dates are unreliable → affects planning and trust.
- *Use case:* For demand forecasting and to adjust estimated delivery times shown to customers.

3. Average Delivery Lead Time (days)

```
Avg Lead Time (days) = AVG( delivered_customer_date - delivered_carrier_date )
```

Explanation:
This measures **carrier efficiency** — how long it takes once the package leaves the seller/warehouse until it reaches the customer.

- Short lead time = good carrier performance.
- Long lead time = transit delays, weak courier network.
- *Use case:* Evaluate and benchmark carriers or 3PL providers.

4. Order Cycle Time (hours)

```
Cycle Time (hrs) = AVG( delivered_customer_date - purchase_timestamp )
```

Explanation:
This is the **end-to-end speed** from order placement → delivery. It includes:

- Payment approval
- Seller handling
- Dispatch + Transit
- Delivery to customer

It shows the **total customer wait time**.

- Short cycle times = happier customers + less chance of cancellation
- Long cycle times = frustration, higher churn.
- *Use case:* Benchmark product categories (e.g., electronics vs. fashion).

5. Stage Breakdown (days)

```
Processing Time = approved_at - purchase_timestamp
```

```
Dispatch Time   = delivered_carrier_date - approved_at
Transit Time    = delivered_customer_date - delivered_carrier_date
```

Explanation:
Breaking down the order cycle helps us **see where the bottleneck is**:

- **Processing Time** → How long sellers take to prepare the order.
- **Dispatch Time** → How quickly orders move from approval to carrier pickup
- **Transit Time** → Carrier speed to final delivery.
- *Use case:* Diagnose issues. For example:
    - Long processing = sellers need SLA training.
    - Long transit = negotiate with carriers.
    - Long dispatch = Olist internal delays.

Data Understanding

Overview of the Olist Dataset

The Olist dataset represents real transactional data from Brazil's largest e-commerce marketplace, covering approximately 100,000 customer orders between 2016 and 2018.

It comprises **nine interrelated tables**, each capturing a different part of the e-commerce journey—from order placement to customer feedback.

Dataset	Description	Primary Keys / Joins	Relevance to Logistics

orders_dataset	Core transactional table containing order IDs and key timestamps (purchase, approval, shipping, delivery, estimated delivery).	order_id	Central for logistics KPIs (cycle time, OTD %, DPA %, lead time).
order_items_dataset	Details of each item within an order (product, seller, price, freight).	order_id, order_item_id	Allows per-item shipment analysis and freight cost evaluation.
customers_dataset	Unique customer identifiers and geographic codes.	customer_id, customer_unique_id	Enables regional delivery performance analysis.
sellers_dataset	Seller profiles including zip code prefixes and locations.	seller_id	Helps identify seller-level handling or dispatch delays.
geolocation_dataset	Zip code-level latitude/longitude mapping.	geolocation_zip_code_prefix	Supports geospatial aggregation and distance estimation.
order_payments_dataset	Payment type, installments, and payment value.	order_id	Useful for correlating payment approval times with order cycle duration.

`order_reviews_dataset`	Customer ratings, comments, and review timestamps.	`order_id`	Connects delivery performance to satisfaction levels.
`products_dataset`	Product attributes such as category, dimensions, and weight.	`product_id`	Enables category-wise delivery benchmarking.
`order_translations_dataset`	Optional mapping for category translation.	`product_category_name`	Enhances readability for dashboards.

Relationships

All datasets are linked via unique identifiers such as `order_id`, `customer_id`, `seller_id`, and `geolocation_zip_code_prefix`. These relationships form the foundation for the **galaxy schema** used later in the Medallion architecture:

Key Features

Understanding timestamp columns was critical to interpreting Olist's logistics process.
The major time fields are:

Timestamp Field	Meaning	Event Type
`order_purchase_timestamp`	Customer placed the order	Customer action
`order_approved_at`	Payment confirmed	Platform action
`order_delivered_carrier_date`	Package handed to carrier	Seller/carrier interface
`order_delivered_customer_date`	Package received by customer	Delivery confirmation

`order_estimated_delivery_date`	System-predicted delivery date	Promise indicator

These timestamps allow calculation of all major logistics metrics (processing time, lead time, OTD %, DPA %, etc.) and serve as the backbone of the **Gold layer analytics**.

Data Volume and Coverage

- **Orders:** ~100,000 unique transactions
- **Customers:** ~90,000+ customers across **27 Brazilian states**
- **Sellers:** ~3,000 unique sellers
- **Reviews:** ~100,000 reviews (1–5 stars)
- **Products:** ~3,000 product categories

The time period spans **September 2016 to October 2018**, giving enough temporal coverage to detect **seasonality and long-term logistics trends**.

Initial Data Quality Observation with YData-Profiling

As a first step, we used Pandas Profiling to quickly scan all the datasets. Several **data quality issues** were observed:

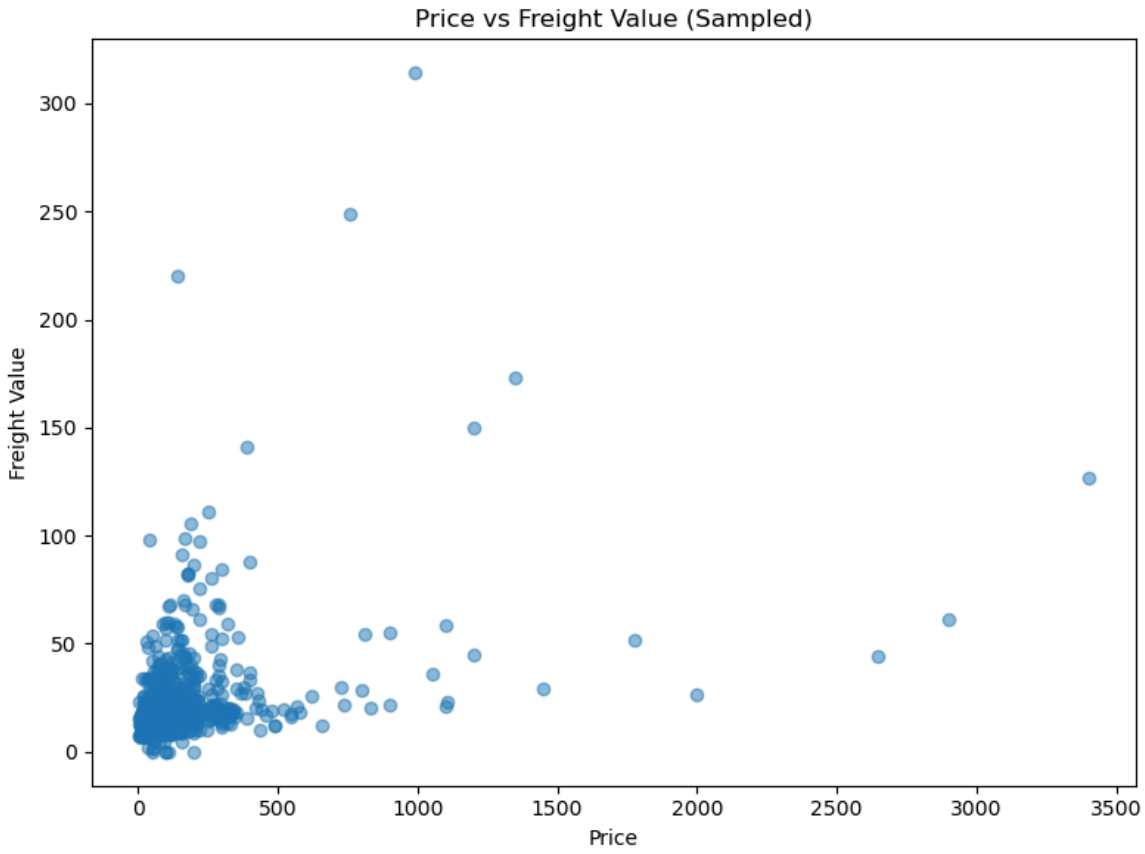
- Missing values
- Data inconsistencies
- Duplicate entries

Preliminary Data Profiling Insights

*Note: The findings below are based on preliminary data. **Final accuracy may vary** once quarantined or invalid records are excluded during the data cleaning stage.*

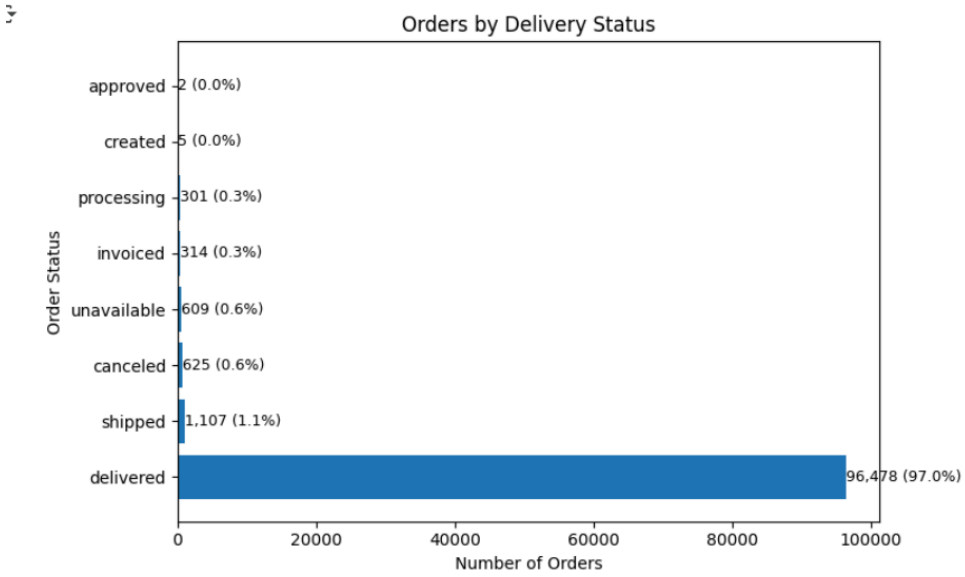
Manual exploratory analysis of the raw datasets using matplotlib revealed some insights:

Order Items

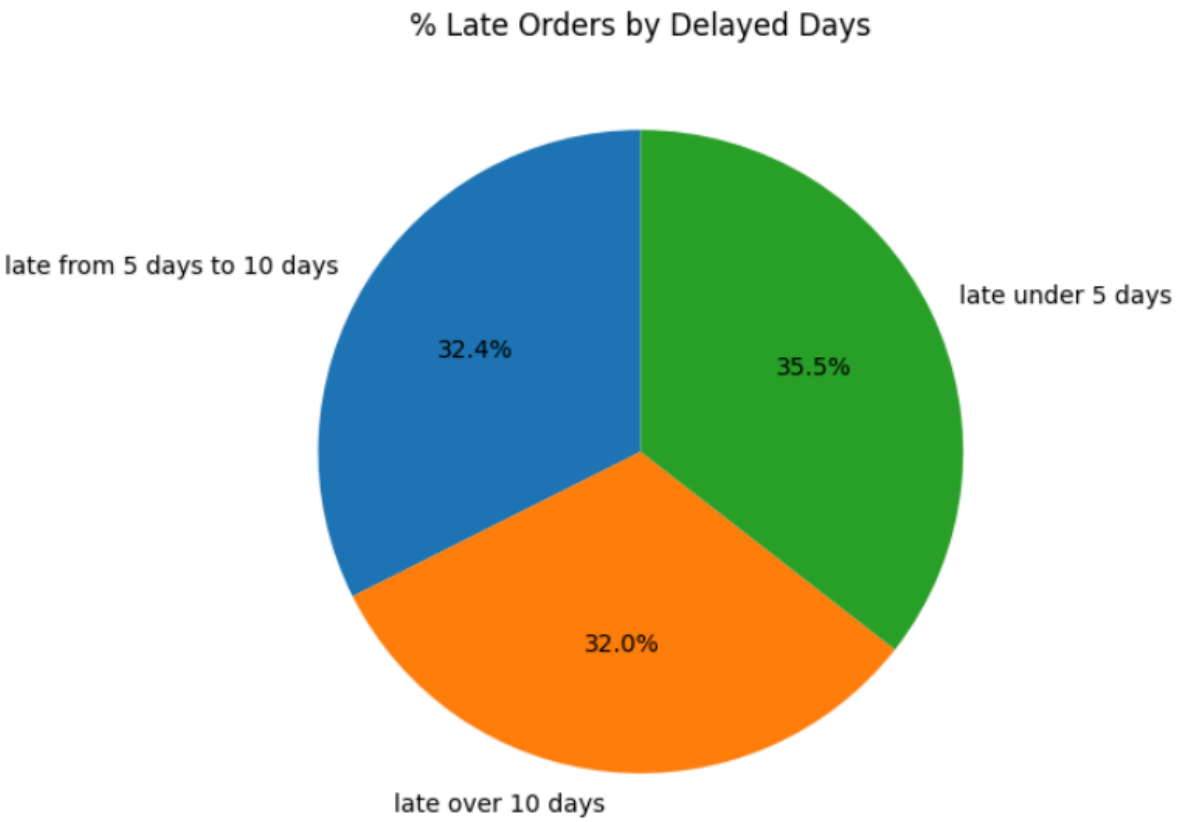


The scatter plot shows a weak relationship between product price and freight value, indicating that shipping cost is influenced more by factors like distance or weight than by the item's price.

Orders

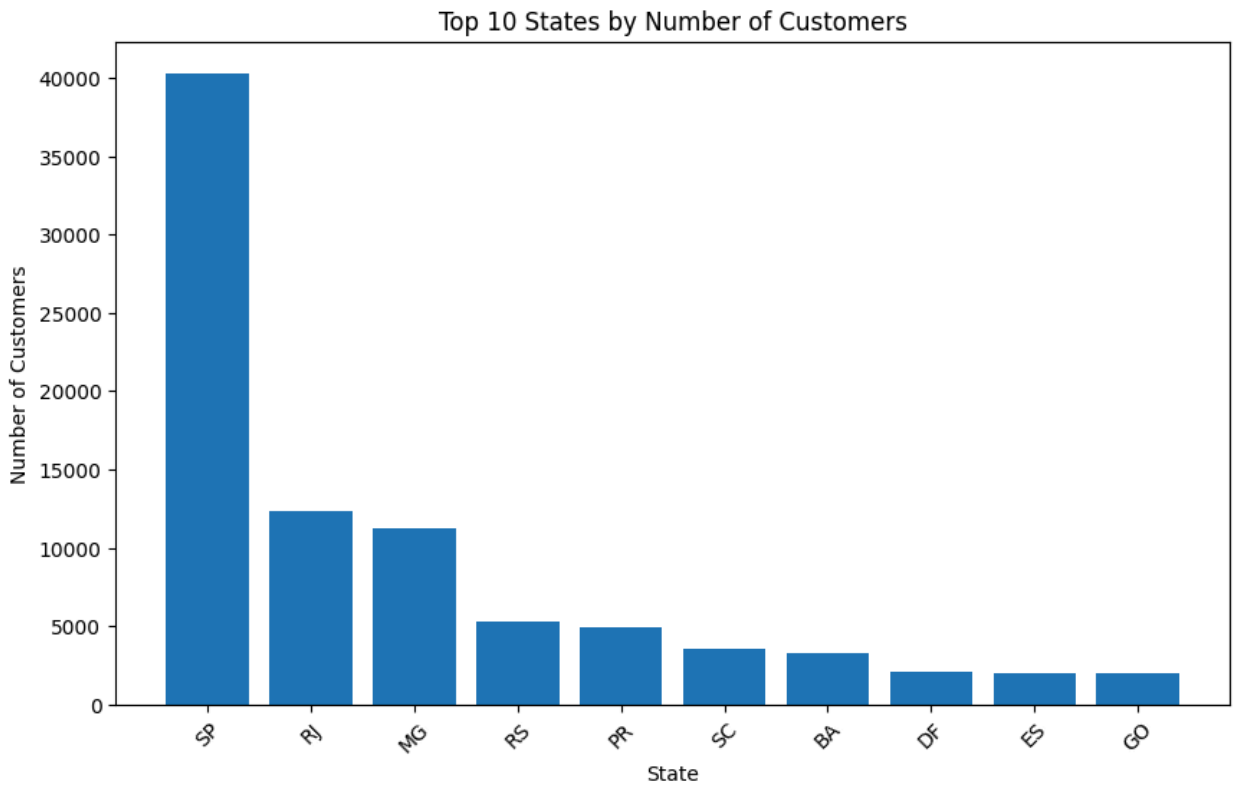


The vast majority of orders (**97%**) were successfully **delivered**, with only a small fraction (**~3%**) canceled, unavailable, or still in process — indicating **high fulfillment efficiency**, though further analysis of the small undelivered group could uncover potential operational gaps.

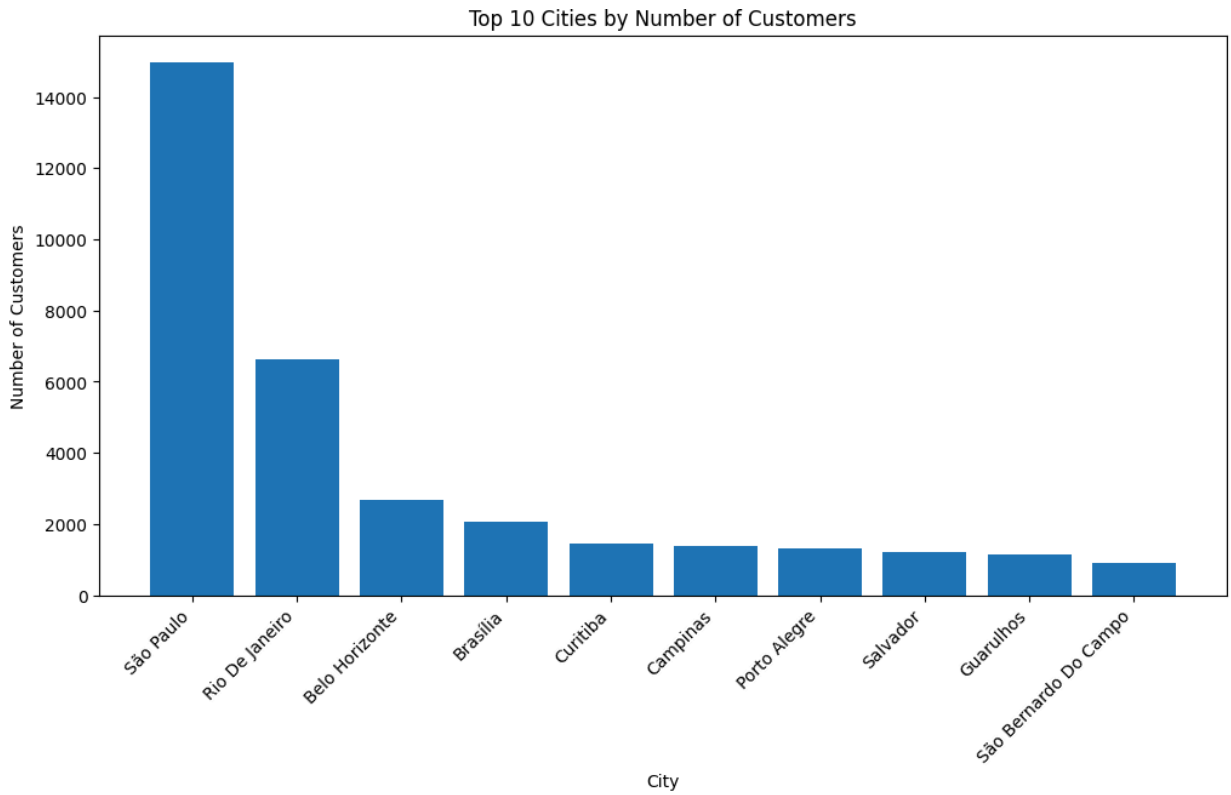


Most late deliveries (35.5%) were delayed by **under 5 days**, while roughly **one-third** were late by **5–10 days** and another third by **over 10 days** — showing that **minor to moderate delays** are common, but **a significant share experience long delays**, signaling room for logistics improvement.

Customers



Customer distribution is highly concentrated in **São Paulo (SP)**, which alone accounts for the vast majority of customers, followed by **Rio de Janeiro (RJ)** and **Minas Gerais (MG)**. This suggests that Olist's customer base is predominantly urban and centered in Brazil's most populous southeastern states.



At the city level, **São Paulo** leads by a large margin, followed by **Rio de Janeiro** and **Belo Horizonte**, reflecting the same regional dominance seen at the state level. These cities represent the main logistics and demand hubs in Brazil's e-commerce market.

Customer Uniqueness

Only 2,997 (3.12%) placed repeat orders — showing that the dataset largely represents one-time purchasers, typical of a marketplace model focused on first-time transactions rather than long-term customer retention.

Data Understanding Implications on ETL pipeline decisions

Our understanding of data characteristics directly informed ETL pipeline decisions:

- Normalized timestamp formats and ensured chronological ordering.
- Applied referential integrity constraints to maintain consistent keys.
- Introduced basic data quality checks (null detection, date logic validation) in Silver layer.

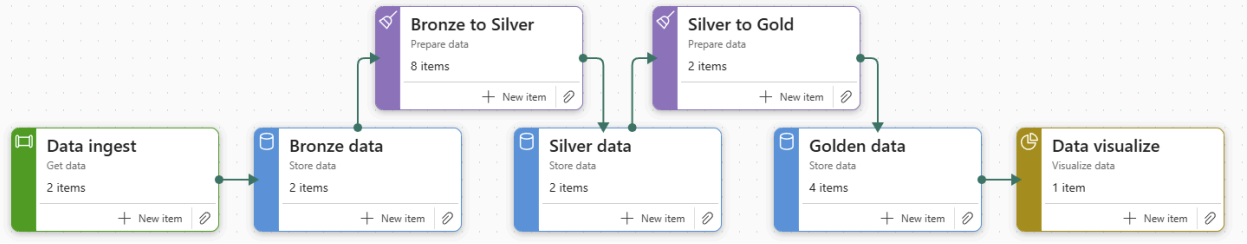
- Created **derived metrics columns** (e.g., `processing_time_days`, `transit_time_days`, `is_on_time`) in Gold layer for analytics.

By combining business and data understanding, we ensured the transformed data aligns with **real logistics logic**, not just technical structure.

System Design

Architecture Overview

The project implements a Medallion Architecture within Microsoft Fabric to organize and process the e-commerce platform's datasets (orders, customers, payments, products, sellers, reviews, etc.) efficiently from raw ingestion to analytics.



Medallion Layers

Layer	Role	Description	Format
Bronze	Raw ingestion	Landing zone for all 9 CSVs from Olist dataset. Stores structured, semi-structured, and unstructured data in original format for auditability.	CSV
Silver	Refined / Cleaned data	Data is cleaned, deduplicated, validated, and integrated. Joins across multiple datasets for consistent organizational access.	Parquet
Gold	Aggregated / Analytics-ready	Data modeled for business intelligence in Galaxy Schema (Fact + Dimension tables). Enables KPI tracking and reporting in Power BI.	Parquet

Purpose: Each layer progressively improves data quality, enabling scalability, performance, and reliable analytics.

Technologies and Tools

Component	Technology	Purpose
Data Lakehouse	Microsoft Fabric Lakehouse	Built on Delta Lake format with ACID support. Combines flexibility of a data lake and query performance of a warehouse. Enables unified analytics on structured and unstructured data.
Data Transformations	Notebooks (PySpark)	Used for complex, large-scale transformations across Silver and Gold layers. Offers better performance and flexibility than low-code Dataflows.
Data Orchestration	Notebooks (PySpark)	Used for manual orchestration of ingestion, transformation, and loading across Bronze → Silver → Gold. Handles
Analytics & BI	Power BI (Direct Lake Mode)	Builds interactive dashboards directly connected to the Lakehouse for near real-time insights.

Design Considerations

Data Volume:
Determines storage type, compute resources, and performance tuning.

Transformation Complexity:
Decides between low-code Dataflows or code-based PySpark Notebooks.

Frequency of Movement:
Batch (daily) vs. near real-time ingestion.

Team Familiarity:
Leverages team's existing skills for productivity and maintainability.

Data Pipeline Workflow

1. Create Lakehouse – Define Bronze, Silver, and Gold layers.
2. Ingest Data (Bronze) – Load 9 CSV datasets (orders, customers, products, reviews, etc.).
3. Transform Data (Silver) –
    a. Handle nulls, duplicates, and schema mismatches
    b. Validate integrity and consistency

4. Model Data (Gold) –
    a. Build Galaxy Schema (Fact + Dimension tables)
    b. Prepare KPI metrics for Power BI
5. Automate with Pipelines – Schedule daily refreshes, run data validation checks.
6. Load to Power BI Semantic Model – Expose datasets via Direct Lake mode for dashboarding.

ETL Design Parameters

Parameter	Choice / Logic
Data Volume	Low; batch ingestion insufficient
Transformation Complexity	Medium–High; multiple joins and aggregations
Update Frequency	Manual refresh, (Gold → Power BI)
Update Pattern	Append for transactional data, overwrite for dimension updates
Schema Design	Galaxy Schema for optimized Power BI performance
Fact Table Type	Transactional (orders, payments, reviews)

Silver Layer - Data Cleaning

General Cleaning Tasks

- **Removed unnecessary whitespace** and **standardized text casing**
  → e.g., `" São Paulo "` → `"sao paulo"`, `"sp "` → `"SP"`.
- **Removed accents and special characters** using `unidecode()` for consistent joins.
- **Standardized ZIP prefixes** to 5-digit numeric format (`"958"` → `"00958"`).
- **Dropped duplicates** based on primary keys (`customer_id`, `seller_id`, `order_id`, etc.).
- **Validated schema and data types** — cast all date, numeric, and string fields explicitly.

- **Checked nulls and blanks** in key columns; retained only when logically acceptable.
- **Wrote assertions** to confirm record counts and unique keys remained stable post-cleaning.

Critical Data Quality Fixes

Area	Issue	Fix	Impact
Orders	**Timestamps were out of order (e.g., delivered before purchase).**	Enforced sequence: `purchase < approved < delivered ≤ estimated`; quarantined invalid rows.	Ensures delivery metrics (OTD, DPA) reflect real event timelines.
Order Items	Broken links between orders, products, and sellers.	Validated all foreign keys; **orphan rows quarantined for traceability.**	Prevents missing data when joining for fulfillment metrics.
Geolocation	Same ZIP prefix mapped to multiple city/state variants.	**Used frequency-based rule to keep the most common mapping per ZIP.**	**Deterministic ZIP-to-city linkage for Customers and Sellers.**
Customers & Sellers	Mixed casing and accented city/state names; malformed ZIPs.	Standardized format, trimmed text, fixed ZIP prefixes.	Consistent joins with Geolocation and Orders.
Reviews	Portuguese comments and timestamp inconsistencies.	Translated text, added sentiment scores, and quarantined bad dates.	Enables sentiment analysis and time-aligned feedback tracking.

Data Validation & Quarantine Strategy

- Implemented **referential integrity checks** (foreign-key matching across Silver tables).
- Created **quarantine tables** for invalid or orphaned records instead of deleting them.
- Verified **chronological order** and **logical consistency** in all event-based datasets.

- All assertions logged and validated before data promotion to Silver.

Result

- All Silver tables are **clean, standardized, and referentially consistent**.
- Key data quality issues (timestamps, broken joins, invalid amounts) were resolved or quarantined.
- Each dataset now supports **accurate, reproducible metrics** in the Gold layer.
- The pipeline maintains **traceability** — no silent data loss, all re-ingestable if corrected upstream.

Gold Layer

The **Gold layer** represents the **analytics-ready data warehouse** designed for **business consumption and dashboarding (Power BI)**.

It builds on the **Silver layer** (cleaned data) by **modeling facts and dimensions** into a **Galaxy Schema** that supports cross-domain analysis.

Key goals:

- Transform transactional Silver data into **aggregated, denormalized analytical structures**.
- Implement **Kimball-based design** (conformed dimensions, surrogate keys, clear grain).
- Embed **business logic and KPIs** for end-to-end delivery lifecycle tracking.
- Enforce **data quality validations** before serving data to dashboards.

This architecture ensures a **single source of truth** and supports scalable **self-service analytics** across logistics, operations, and customer experience teams.

Key Transformations and Business Logic

`dim_date`

- Generated **continuous date range** spanning 2 years before and 3 years after data coverage for forecast support.
- Derived **calendar attributes**: year, quarter, month, day of week, weekend flag.
- Integrated **Brazil national holidays** using `holidays` package.
- Validated:
    - Unique `date_key` (`yyyyMMdd`).
    - Continuous range coverage.
    - Logical consistency (`is_weekend` aligned with weekday numbers).
    - No nulls in key fields.
      Result: **100% coverage**, **no duplicates**, **audit-ready dimension**
      nb_gold_up

`dim_product`

- Selected only **analytics-relevant columns**: category, size, weight.
- Created **surrogate key** using deterministic `xxhash64()` on `product_id`. Hash is used because **collisions observed when using monotonically_increasing_id.**
- Validated:
    - Surrogate and natural key uniqueness.
    - No missing product IDs or categories.
- Cached dimension for reuse in joins (optimization).

`dim_customer`

- Joined customers with geolocation (via ZIP code prefix) using **broadcast joins.**
- Created `customer_key` with `xxhash64(customer_id)`.

- Validated ZIP coverage (≈99.7%).
- Preserved nulls for missing coordinates (no imputation to maintain data integrity).

`dim_seller`

- Similar process as customers: joined with geolocation, created `seller_key`, ensured referential integrity.

  Result: Seller-level view for seller performance and distance-based insights.

`fact_order_fulfillment` (Primary Fact)

- **Grain:** One row per order (accumulating snapshot).
- Derived **foreign keys** for each milestone date:
  - `purchase_date_key`, `approved_date_key`, `carrier_date_key`, `delivered_date_key`, `estimated_date_key`.
- Joined with `dim_customer` to include `customer_key`.
- Computed **delivery lifecycle metrics**:

Metric	Formula	Meaning
`processing_time_days`	approved – purchase	Time to approve order
`dispatch_time_days`	carrier pickup – approved	Time to hand over to carrier
`transit_time_days`	delivered – carrier pickup	Time in transit
`lead_time_days`	delivered – purchase	End-to-end cycle time
`delivery_variance_days`	delivered – estimated	Delay or early delivery
`is_on_time` / `is_late`	Boolean	SLA flags for delivery promise accuracy

-

- Applied **business logic to prevent false positives**:
    - Computed durations only if both timestamps exist.
    - SLA flags only when order is delivered.
- Validation:
    - No negative durations.
    - No duplicate `order_id`.
    - Referential integrity for all date and customer foreign keys.
      ✅ Result: Reliable foundation for **On-Time Delivery (OTD%)**, **Delivery Variance**, and **Lead Time** KPIs
      nb_gold_up
      .

fact_review

- Joined reviews with fulfillment data to link satisfaction (`review_score`) with delivery accuracy (`is_late`, `delivery_variance_days`).
- Created surrogate `review_key` from `review_id`.
- Derived flag `is_low_rating = 1 if review_score ≤ 2`.
  ✅ Result: Enables analysis of **delivery performance vs. customer sentiment** correlation

Data Quality & Validation Strategy

Each dimension and fact was validated via assertions:

- **Primary key uniqueness** (no duplicates).
- **Null checks** on critical columns.
- **Row count validation** (no data loss).
- **Referential integrity** between FKs and PKs.
- **Logical consistency** (e.g., weekend logic, positive duration checks).

PySpark Performance Optimization

- **Column Pruning** – Selected only **required columns** using .select() to reduce data shuffle and serialization overhead.
- **Broadcast Joins** – Used broadcast() for small lookup or dimension tables (e.g., Dim_Date, Dim_Customer) to **prevent expensive shuffles when joining with large fact tables.**
- **Cache and Persist** – Applied .cache() strategically for **reused intermediate DataFrames, avoiding redundant recomputations.**

Data Modelling

Approach

The dimensional design modelling process can be effectively broken down into four steps:

- Understanding the business process.
- Declaring the grain.
- Identifying the dimensions.
- Identifying the facts.

Galaxy Schema

We implemented a **Galaxy Schema**, a data warehouse design that extends the traditional **Star Schema** by connecting **multiple fact tables** through **shared conformed dimensions** such as **Customer**, **Order**, and **Date**.

Each fact table represents a distinct **business process**, while the conformed dimensions ensure that all facts use the **same, consistent business definitions**.

Conformed dimensions are central to this design. They allow different fact tables to **use the same reference points** for analysis—for example, the same definition of a customer, the same date hierarchy, or the same order identifiers. This consistency enables **cross-process analysis** (e.g., comparing **delivery performance** from Fact_OrderFulfillment with **customer satisfaction** from Fact_Reviews) **without reconciliation or data duplication.**

The Galaxy Schema is unique because it supports **multiple business processes** within one unified structure. It allows us to scale easily by adding new fact tables—such as **Payments** or

Returns—without redesigning the entire model. By using conformed dimensions, we maintain a **single version of truth**, ensure **data consistency**, and **reduce redundancy** across the warehouse.

Although this design introduces slightly more **complexity and maintenance**, it delivers substantial advantages in **scalability**, **flexibility**, and **end-to-end insights**, making it a robust foundation for our logistics analytics model.

We implemented a **Galaxy Schema**, which is a data-warehouse design that builds on the traditional **Star Schema**.

Instead of having just one fact table, the Galaxy Schema connects **multiple fact tables** through **shared conformed dimensions** like **Customer**, **Order**, and **Date**.

This approach lets different fact tables **"speak the same language"** by using common references.

It allows us to analyze how separate business processes relate to each other—for example, how **delivery performance** affects **customer satisfaction**—while keeping each table focused and accurate.

The Galaxy Schema is unique because it supports **cross-process analysis and can easily grow as new business areas are added.**

By using shared dimensions, we maintain a **single version of truth**, ensure **data consistency**, and reduce redundancy across the warehouse.

Although this design is slightly more complex to maintain, it offers clear advantages in **scalability**, **flexibility**, and **end-to-end insights**, making it a strong foundation for our logistics analytics model.

Pros vs Cons of Galaxy Schema Warehouse Design

Advantages	Disadvantages / Challenges
1. Supports multiple business processes Different facts (orders, payments, reviews) can coexist in one model.	**1. More complex relationships** Joins and ETL logic become harder to manage than in a single star schema.

2. Enables cross-domain analysis
You can connect processes (e.g., delivery speed vs. customer satisfaction).

3. Encourages data consistency
Shared dimensions ensure everyone uses the same definition of “Customer,” “Date,” etc.

4. Scalable and modular
Easy to add new business areas (e.g., payments) without redesigning the whole schema.

2. Higher maintenance effort
Changes in shared dimensions affect multiple fact tables.

3. Requires careful governance
Teams must agree on consistent keys and dimension attributes.

4. Performance considerations
Large joins across several facts can slow queries if not optimized.

Consideration - Kimball rules for storing multiple facts

When designing our galaxy schema, our team adhered to **Kimball’s rules for storing multiple facts** within a single fact table. Example of our consideration:

Rule 1 – Same Grain (Level of Detail)

Facts can only live together if they’re recorded at the **same level of detail**—that is, they describe events using the same dimensions.

- `Fact_OrderFulfillment` records information at the **order level** — **one row per order**, tracking key timestamps such as purchase date, dispatch date, and delivery date.

- `Fact_Reviews` records information at the **review level** — **one row per customer** review, containing the rating, comment, and sentiment.

Since one order can have **multiple reviews** or **no review at all**, these two facts are not aligned at the same grain.

Lesson: Different level of detail = must remain in separate fact tables.

Rule 2 – Simultaneous Occurrence (Business Process Timing)

Facts can only be stored together if they happen **at the same time** as part of the **same business process**.

- Fulfillment happens **during** the logistics process — when the order is being shipped and delivered.

- Reviews happen **after** the order is delivered — during the customer-feedback process.

Because these two events occur at different times and belong to distinct processes, they are **not simultaneous**.

Hence, **different process and timing = store separately.**

Our Decision

Since both rules would be violated if the data were combined, our team modeled them as two separate fact tables:

- `Fact_OrderFulfillment` — captures operational performance (delivery speed, lead time, delays).
- `Fact_Reviews` — captures post-delivery customer experience (ratings, sentiment, feedback).

Both tables are linked through **shared dimensions** such as `Dim_Order`, `Dim_Customer`, and `Dim_Seller`.

This structure keeps each table clean and accurate while still allowing cross-analysis, such as **"Does late delivery lead to lower review scores?"**

Consideration: Slowly Changing Dimensions (SCD)

In real-world warehouses, attributes change over time.
A seller moves warehouse (SP → RJ).
A customer relocates.
A product gets reclassified.

SCD handles this history:
Type 1: overwrite old value.
Type 2: keep history with effective dates.

Dimension	Change Over Time?	Need SCD?	Why

DimCustomer	No (location tied to each order)	❌	Each order already stores customer location
DimSeller	No (one fixed record per seller)	❌	No warehouse history in dataset
DimProduct	No (category & attributes static)	❌	Just translations + product metadata
DimDate	nO	❌	Dates don’t change
DimReview	No (reviews fixed once submitted)	❌	Review score/comment stable
DimGeolocation	No (zip mapping static)	❌	Lat/long doesn’t change

The Olist dataset is a **frozen historical snapshot**.It already ties orders to the right customer/seller/product at the time of purchase. There are no “slowly changing” attributes captured.

Hence, we do not implement SCDs for this project.

Fact Types

Additive Facts

- **Definition:** Can be summed across **all dimensions**, including time.
- **Rule of thumb:** Safe to `SUM()` everywhere.
- **Olist examples:**

    - `price` (FactOrderItems) → total sales.
    - `freight_value` (FactOrderItems) → total freight charges.

Semi-Additive Facts

- **Definition:** Can be summed across some dimensions (e.g., product, seller, region), but **not across time**.
- **Rule of thumb:** OK to sum by entity, but not over dates.
- **General examples:** account balances (sum across accounts, not across months)
  Data Warehouse Fundamentals for…
  .
- **Olist examples:**
    - `processing_time`, `dispatch_time`, `transit_time` (FactOrders).
        - You can average them across sellers or categories.
        - But summing across months makes no sense.

Non-Additive Facts

- **Definition:** Cannot be summed meaningfully at all. Must use **average, min, max, or ratio**.
- **Rule of thumb:** Never use `SUM()`, use other aggregations.
- **General examples:** margins, percentages, ratios
- **Olist examples:**
    - `review_score` (FactReviews) → use `AVG()`.
    - `delivery_time_days` (FactOrders) → use `AVG()` or `MAX()`.
    - `delay_days` (FactOrders) → use `AVG()` or `MAX()`.

    - `on_time_flag` (0/1) → calculate `%` on-time, not sum raw values.

Rule of Thumb

- **Additive = "Always SUM"** → prices, freight, payments.
- **Semi-Additive = "SUM by entity, not by time"** → stage durations.
- **Non-Additive = "Never SUM, use AVG/MIN/MAX"** → review scores, delays.

Fact Tables Types

Transaction Fact Table

- **One row = one event.**
- Lowest level of detail.
- **Think:** *"Every time something happens, add a row."*
- **In Olist**
    - Each review → `FactReviews`

Accumulating Snapshot Fact Table

- **One row = one process that goes through stages.**
- Updated as the process progresses.
- **Think:** *"Track the whole life cycle in one row."*
- **In Olist**
    - Each order, from purchase → approved → shipped → delivered → promised → `FactOrders`

Periodic Snapshot Fact Table (Not implemented)

- **One row = one time period (day, week, month).**
- Captures summary at that point in time.

- **Think:** *“Take a photo of the KPIs every day/week.”*

- **In Olist (don’t think needed here since can use GROUP BY)**
    - Daily delivery snapshot → total orders, % on time, avg delay.

Fact Tables (to edit)

FactOrders (Accumulating Snapshot Fact Table)

- **Grain:** one row per order.

- **Why?** An order goes through stages (*purchase → approved → shipped → delivered*). We track the **elapsed time** between stages.

Columns

- `order_id`

- `customer_key` (FK → DimCustomer)

- `order_status`

- `order_purchase_timestamp`

- `order_approved_at`

- `order_delivered_carrier_date`

- `order_delivered_customer_date`

- `order_estimated_delivery_date`

Example Row

order_id	purchase	approved	shipped	delivered	promised
A1	2017-01-01 10:00	2017-01-01 12:00	2017-01-03 09:00	2017-01-05 15:00	2017-01-06 23:59

Derived Metrics

- Days purchase → approval: 0.1 days (2 hrs)
- Days approval → ship (seller handover to courier): 1.9 days
- Days ship → delivered: 2.25 days
- Total lead time: 4.2 days
- Delay days: -1 (delivered early)
- On-time flag: 1

Business Use

- “What % of orders are on time?”
- “Average cycle time across orders?”
- “Which stage is the slowest?”

FactOrderItems (Transaction Fact Table)

- **Grain:** one row per order item.
- **Why?** Orders have multiple items; we need detail at product level.

Columns

- `order_id` (FK → FactOrders)
- `order_item_id`
- `product_key` (FK → DimProduct)
- `seller_key` (FK → DimSeller)
- `shipping_limit_date`

- `price`
- `freight_value`

Example Rows

order_id	item_id	product_key	seller_key	price	freight_value
A1	1	P123	S1	100.00	15.00
A1	2	P456	S2	50.00	8.00

Business Use

- “Which categories/products have long delivery times?”
- “Are heavier/more expensive items delayed more?”
- “Which sellers ship the highest volume?”

FactReviews (Transaction Fact Table)

- **Grain:** one row per review.
- **Why?** Each review is a customer feedback event tied to an order.

Columns

- `review_id`
- `order_id` (FK → FactOrders)
- `review_score`

- `review_comment_message`
- `review_creation_date`
- `review_answer_timestamp`

Example Rows

review_id	order_id	score	comment
R1	A1	5	“Great, on time!”
R2	A2	1	“Arrived 2 days late.”

Business Use

- “% of low ratings linked to late deliveries?”
- “Average score for on-time vs late deliveries?”
- “Which sellers/categories get the worst reviews?”

Dimension Tables

DimCustomer

customer_key	customer_id	city	state	zip_prefix
C1	c123	São Paulo	SP	01001

DimSeller

seller_key	seller_id	city	state	zip_prefix
S1	s567	Rio de Janeiro	RJ	22041

DimProduct

product_key	product_id	category	weight_g	length_cm
P123	p999	electronics	500	20

DimDate

Instead of recalculating `EXTRACT(YEAR FROM date)` every time, we just join to `DimDate` and use `year`.

date_key	date	day	month	year	week_of_year
D20170101	2017-01-01	1	1	2017	1

DimReview

review_key	review_id	score	comment_title	comment_message
R1	r123	5	Fast delivery	“Delivered before promised date.”

Semantic Model

How to Test Semantic Model Relationships in Power BI

1. Row Count Sanity Checks

- Compare the number of rows in fact tables with and without dimension joins.
- Example:
    - SQL: `SELECT COUNT(*) FROM gd_fact_order_fulfillment`
    - Power BI visual: count of `order_id`.
    - They should match (unless filtering intentionally).

2. Foreign Key Coverage

- Test if all FK values exist in the related dimension.

SQL version:

```
 SELECT COUNT(*)
FROM gd_fact_order_fulfillment f
LEFT JOIN gd_dim_customer d
  ON f.customer_key = d.customer_key
WHERE d.customer_key IS NULL;
```

-
- If you get non-zero results → missing dimension keys → relationship might drop rows in Power BI.

3. Bidirectional vs Single Direction Filter Flow

- Test by applying slicers in Power BI:
    - Example: Filter by `state` in `dim_customer`.

    - Expected: All measures in fact table should update correctly.

- If blanks or duplicate totals appear → relationship is wrong (or needs a different direction).

4. Role-Playing Date Relationships

- In fact tables with multiple date FKs (purchase, approval, delivered):
    - Check each relationship explicitly with `USERELATIONSHIP()` in DAX.

Example:

```
 Orders by Delivered Date =
CALCULATE(COUNTROWS(gd_fact_order_fulfillment),

USERELATIONSHIP(gd_fact_order_fulfillment[delivered_date_key],
dim_date[date_key]))
```

    -

Compare with SQL:

```
 SELECT delivered_customer_date, COUNT(*)
FROM sl_orders
GROUP BY delivered_customer_date;
```

If numbers differ → relationship mapping is wrong.

5. Filter Propagation Testing

- Build simple test visuals:
    - Count of `order_id` by `dim_date[month]`.
    - Count of `order_id` by `dim_customer[state]`.
- Cross-check against raw SQL group-bys.

- If totals mismatch → either wrong join key or inactive relationship.

6. Edge Case Testing

- Pick a **single order_id** and trace it:
    - In SQL: join fact → customer → date → seller → geolocation.
    - In Power BI: filter to that order_id and see if attributes line up.
- This “unit test” approach often reveals mis-mapped keys (e.g., state attached to wrong FK).

Testing Correctness of Semantic Model

Your semantic model relationships are correct when:

1. Fact totals = SQL totals.
2. Filters from dimensions propagate correctly.
3. Role-playing dates produce the same results as SQL group-bys.
4. No unexpected blank or duplicate values appear in visuals.

BI Dashboard

Visualization & Storytelling Standards

Goal

Turn metrics into **insights that drive action**, not just charts that show data.
Every visual must answer:

“What’s happening?” → “Why does it matter?” → “What should we do next?”

3-Layer Framework — The “So What → Why → What Next”

Every finding needs **three parts**:

Layer	Example
The Data	“18% of deliveries were late in 2018.”
The Impact	“These caused 67% of 1–2★ reviews and R$2.3M in lost customers.”
The Action	“Target 15 slow sellers; auto-alert when dispatch >24 hrs.”

The “So What” Test

Before finalizing any chart, ask:

> **“Would the CEO know what to do next after seeing this?”**
> If not → refine your insight or link it to business impact.

Dashboard Structure

Page	Content
Landing Page	4–6 KPI cards, 1–2 key charts, traffic light indicators. **No scrolling.**
Detail Pages	1 page per category (Delivery, Geo, Seller, Customer). Max 3–4 visuals each.
Annotations	Add short text directly on charts: “Black Friday: +180% volume, OTD ↓ to 72%.”

Business Recommendations Framework

Priority	Action	Owner	Timeline	Expected Impact	Tracking Metric
HIGH	Auto-alerts for sellers >48hr dispatch	Ops	30 days	↓ Dispatch delays 20%	Avg Dispatch Time

3 Time Horizons

- **Immediate (0–30d):** Quick wins (e.g., seller alerts).
- **Strategic (30–90d):** Root causes (e.g., new rural logistics partners).
- **Policy (90d+):** System changes (e.g., dynamic delivery promises).

Include ROI

Investment: R$150K → *Benefit:* R$420K saved → **ROI: 180% in Year 1**

Presentation Structure

1. **Headline:** “Delivery improved +11 pts, but 15 sellers cause 60% of complaints.”
2. **Show Proof:** 3–4 visuals with clear takeaways.
3. **Close with Roadmap:** Prioritized actions, owners, and metrics.

Definition of Done

A visualization is done when it meets all of the following:

1. Data Accuracy & Validation

- Values are calculated using the correct fact/dimension tables and follow the agreed KPI definitions.

- Aggregations (SUM, AVG, COUNT, %) are applied correctly.
- **Results are validated against source queries or sample data checks.**

2. Clarity of Business Meaning

- The visual has a precise, self-explanatory title (e.g., *"On-Time Delivery % by Region"*).
- Axis labels, legends, and units (days, hours, %, counts) are consistent and unambiguous.
- Metrics shown align with business terminology (OTD, DPA, Cycle Time, etc.).

3. Design & Usability

- The chart type is appropriate for the metric (e.g., KPI card for % targets, line chart for trends, stacked bar for stage breakdowns).
- Conditional formatting is applied for thresholds (e.g., SLA ≥95% = green, otherwise red).
- Slicers and filters work correctly; visuals respond as expected to cross-filtering.

4. Performance & Reliability

- Visuals load within acceptable time (<5s under normal conditions).
- No redundant or heavy DAX queries that could be simplified.
- **Dataset relationships and filters are stable (no unexpected blank rows or duplication).**

Logistics KPIs

Derived Metrics (KPI Pre-Computations)

The following time-based metrics and flags are derived in the **Fact_OrderFulfillment** table to support delivery-performance analysis:

Metric / Flag	Definition	Formula / Logic	Purpose
Processing Time	Duration between order placement and approval.	`order_approved_at - order_purchase_timestamp`	Measures how long it takes the seller or system to approve an order after it's placed.
Dispatch Time	Duration between order approval and carrier pickup.	`order_delivered_carrier_date - order_approved_at`	Indicates seller's efficiency in preparing and handing over the package to the logistics carrier.
Transit Time	Duration between carrier pickup and customer delivery.	`order_delivered_customer_date - order_delivered_carrier_date`	Captures the logistics carrier's delivery efficiency.
Delivery Lead Time	Total duration from order placement to actual delivery.	`order_delivered_customer_date - order_purchase_timestamp`	Represents the end-to-end customer waiting time.
Delivery Variance	Difference between actual and promised delivery date.	`order_delivered_customer_date - order_estimated_delivery_date`	Measures deviation from promised delivery time.
On-Time Delivery Flag	Indicates if delivery met or beat the promise date.	`1 if order_delivered_customer_date <= order_estimated_delivery_date else 0`	Used for KPI **On-Time Delivery % (OTD%)**.
Late Delivery Flag	Indicates if delivery was delayed.	`1 if order_delivered_customer_date > order_estimated_delivery_date else 0`	Used for KPI **Late Delivery %** and correlation with low review scores.

1. What is the On-Time Delivery (OTD) Rate across all orders?

Definition: % of orders delivered on or before the estimated delivery date
Formula: **OTD % = Orders where (delivered_date ≤ estimated_date) ÷ Total Orders**

What it measures:

- Binary outcome: Did you meet the promise? (Yes/No)
- Only cares about late vs not late
- Treats delivering 1 day early the same as 10 days early

Example:

- Order A: Promised Feb 10, Delivered Feb 8 (2 days early) → Counts as On-Time ✓
- Order B: Promised Feb 10, Delivered Feb 10 → Counts as On-Time ✓
- Order C: Promised Feb 10, Delivered Feb 11 (1 day late) → Counts as Late ✗

Step 1 - Create DAX Measure:

```
On-Time Delivery Rate =
DIVIDE(
    CALCULATE(
        COUNTROWS('gd_fact_order_fulfillment'),
        'gd_fact_order_fulfillment'[is_on_time] = 1
    ),
    CALCULATE(
        COUNTROWS('gd_fact_order_fulfillment'),
        NOT(ISBLANK('gd_fact_order_fulfillment'[delivered_date_key])),
        NOT(ISBLANK('gd_fact_order_fulfillment'[estimated_date_key]))
    ),
    0
) * 100
```

Explanation:

Numerator: Counts orders where On-Time Delivery Flag = 1 (delivered on or before promised date)

Denominator: Counts only orders with **both delivered_customer_date** and **estimated_delivery_date** (filters out incomplete records)

DIVIDE with 0: Prevents division by zero errors

Step 2 - Create Primary KPI Visual

Why KPI Card? Single-value metrics that executives need to understand at a glance are best displayed as large, prominent KPI cards

2. What is the Delivery Promise Accuracy (% delivered on the promised date)?

3. What is the Average Delivery Lead Time (days from purchase to delivery)?

4. What is the Delivery Time Variance (days early/late vs promised)?

5. What is the average Order Cycle Time (purchase → delivery)?

6. How much time is spent in each stage (Processing, Dispatch, Transit)?

7. How has the average delivery time changed over 2016–2018?

Which regions incur the highest delivery costs and delays?

Approach: Calculated average freight value and delivery days per seller state. Plotted states on a scatterplot by cost vs speed.

Seller & Product Analysis

1. Purpose

The Seller & Product Analysis aims to evaluate seller performance, product popularity, and fulfillment efficiency. This helps the business owner make data-driven decisions on which sellers to reward, which products to promote, and where operational improvements are needed.

---

2. Datasets Used

- `olist_sellers_dataset.csv`
- `olist_products_dataset.csv`
- `olist_order_items_dataset.csv`
- `olist_orders_dataset.csv`
- `olist_order_payments_dataset.csv`

---

3. Data Processing Steps (Microsoft Fabric)

Bronze Layer (Raw):

- Uploaded raw CSVs into Microsoft Fabric Lakehouse.
- Ensured schema consistency and verified data types (seller_id, product_id, etc.).

Silver Layer (Cleaned):

- Removed duplicates, handled nulls, standardized date formats.
- Joined tables using keys:
    - `seller_id` (link seller and orders)

    - `product_id` (link products and items)
    - `order_id` (connect payments and orders)
- Calculated total revenue per seller and average delivery delay.

Gold Layer (Analytical):

- Aggregated sales and performance metrics for sellers and products.
- Created Power BI measures to calculate KPIs for visualization.

---

4. Key Performance Indicators (KPIs)

KPI	Formula / Basis	Business Insight
Total Revenue per Seller	SUM(payment_value) grouped by seller_id	Measures sales performance
Average Delivery Delay (Days)	DATEDIFF(order_estimated_delivery_date, order_delivered_customer_date)	Indicates delivery efficiency
Top Product Categories	COUNT(order_id) grouped by category	Identifies high-demand categories
Revenue by Category	SUM(payment_value) by product_category	Shows which products drive revenue
Seller Contribution %	Seller Revenue / Total Revenue	Identifies key sellers

---

5. Power BI Dashboard Visuals

Section	Visual Type	X-Axis	Y-Axis / Value	Purpose
Seller Performance	Bar Chart	Seller ID	Avg. Revenue	Compare seller contribution

Product Popularity	Treemap	Product Category	Order Count	Show top-selling products
Delivery Efficiency	Column Chart	Seller ID	Avg. Delivery Delay	Highlight delays by seller
Seller Contribution	Pie Chart	Seller ID	% of Total Revenue	Show key seller share

---

6. Insights

- A small number of sellers contribute the majority of revenue.
- Certain sellers have consistently longer delivery times.
- Product categories with higher sales volumes also show more late deliveries.

---

7. Recommendations

- Focus on improving logistics for underperforming sellers.
- Promote top-selling categories through marketing campaigns.
- Consider replacing or retraining low-performing sellers.

Review Analysis

1. Purpose

The Review Analysis identifies how fulfillment and product quality affect customer satisfaction. By examining review scores and their correlation with delivery time, the business owner can pinpoint service bottlenecks that impact reputation and retention.

---

2. Datasets Used

- `olist_order_reviews_dataset.csv`
- `olist_orders_dataset.csv`
- `olist_order_items_dataset.csv`
- `olist_order_payments_dataset.csv`

---

3. Data Processing Steps (Microsoft Fabric)

Bronze Layer (Raw):

- Loaded raw CSVs into Microsoft Fabric.

Silver Layer (Cleaned):

- Removed empty reviews and standardized date fields.
- Created calculated columns for delivery delay and review sentiment (Positive/Neutral/Negative).
- Joined review data with orders via `order_id`.

Gold Layer (Analytical):

- Created summarized tables for average review scores, sentiment ratios, and review vs delay correlation.

---

4. Key Performance Indicators (KPIs)

KPI	Formula / Basis	Business Insight
Average Review Score	AVG(review_score)	Measures overall satisfaction

% Negative Reviews	COUNT(review_score ≤ 2) / Total Reviews	Identifies dissatisfaction rate
Avg Review Delay Impact	AVG(review_score) vs delivery delay	Correlation between lateness and ratings
Avg Payment Value per Review	SUM(payment_value) / COUNT(review_id)	Links satisfaction with purchase size
Review Volume Over Time	COUNT(review_id) by Month	Measures engagement trend

---

5. Power BI Dashboard Visuals

Section	Visual Type	X-Axis	Y-Axis / Value	Purpose
Review Sentiment Breakdown	Doughnut Chart	Sentiment (Positive/Negative)	% of Reviews	Overview of satisfaction level
Review Trend Over Time	Line Chart	Month	Avg Review Score	Track satisfaction changes
Review vs Delivery Delay	Scatter Plot	Delivery Delay	Review Score	Correlation insight
Review Distribution	Bar Chart	Review Score (1–5)	Count of Reviews	Sentiment pattern
Negative Review Analysis	Bar Chart	Seller ID	% Negative Reviews	Identify underperforming sellers

---

6. Insights

- Negative reviews increase when delivery delays exceed estimated times.
- Most negative reviews are related to late delivery or poor product quality.
- Sellers with consistent on-time delivery have significantly higher review scores.

---

7. Recommendations

- Prioritize seller training on timely delivery and packaging quality.
- Introduce alerts for potential delivery delays before customer dissatisfaction occurs.
- Monitor reviews monthly to detect early shifts in customer sentiment.

Testing

Functional Testing

Objective:
To ensure all dashboards, data transformations, and visualizations are working as expected and accurately reflect the data processed from Bronze → Silver → Gold layers.

Scope:

- Validate that datasets are correctly transformed and loaded into the Lakehouse.
- Confirm that measures and calculated columns in Power BI return accurate values.
- Ensure filters, slicers, and charts respond dynamically to user interaction.

Test Scenarios:

1. **Data Connectivity Test**
   Verify that all Power BI visuals are connected to the correct Gold layer tables.
   - Expected Result: Visuals update correctly when data in Fabric is refreshed.
2. **Data Accuracy Test**
   Compare values between the Gold layer dataset and dashboard totals.
   - Expected Result: No mismatch between the dataset output and visualization totals.
3. **Filter and Slicer Test**
   Check if applying filters such as "Review Score" or "Seller ID" updates all related visuals.
   - Expected Result: All visuals adjust accurately to filter selections.

4. **Navigation Test**
   Ensure navigation between different dashboard pages (e.g., Seller & Product Analysis, Review Analysis) functions properly.

    - Expected Result: Buttons or tabs link users to the correct dashboard pages.

5. **Visualization Logic Test**
   Validate that charts display the intended data relationships.

    - Example: Doughnut chart correctly shows percentage of late deliveries among negative reviews.

    - Expected Result: Each visual aligns with the defined business logic.

6. **Performance Test**
   Check dashboard loading time and responsiveness.

    - Expected Result: Dashboard loads within acceptable time without lag or data error.

User Acceptance Testing (UAT)

Objective:
To validate that the final Power BI dashboard meets the business requirements of the problem statement: improving on-time delivery and strengthening customer satisfaction.

Scope:

- Conducted from the perspective of a business owner analyzing seller, product, and customer review performance.

- Test cases are aligned with real-world decision-making needs.

Test Scenarios:

☐ **Usability Test**
Verify that dashboard navigation, visuals, and metrics are clear and intuitive.

- ☐ Expected Result: The dashboard is user-friendly and easily interpretable.

☐ **Relevance to Business Needs**
Check whether KPIs (delivery rate, review sentiment, product performance) directly

support decision-making for improving customer satisfaction.

- ☐ Expected Result: The metrics displayed are relevant and actionable.

☐ **Insight Accuracy**
Assess if insights derived from visuals (e.g., top-performing sellers, late delivery trends) reflect accurate data patterns.

- ☐ Expected Result: Insights are logical and consistent with dataset analysis.

☐ **End-to-End Flow Validation**
Confirm that data flows seamlessly from ingestion to visualization (Bronze → Silver → Gold → Dashboard).

- ☐ Expected Result: No broken connections or missing datasets.

☐ **Business Impact Validation**
Ensure the dashboard helps in identifying problem areas (e.g., low review scores linked to delivery delays).

- ☐ Expected Result: The dashboard provides actionable insights for operational improvement.

Future Enhancements

- Move beyond assertions which is not best practice for production systems.
  - Centralized **Quality Log table** for automated tracking.
  - Integration with **Great Expectations** for declarative validation rules.
- Perform integration testing, end-to-end testing
- Build predictive models (E.g. Delivery Delay Prediction) using Azure Machine learning
- Deploy demand forecasting pipelines (Holt-Winters, Facebook Prophet)
- Implement partitioning for large fact tables